Machine Intelligence Terminology & Concepts Reference

Contents

A.1 Course Structure: How Lectures Connect **2**

A.2 Key Terms Dictionary **2**

A.3 L7: Linear Regression **3**

A.4 L8: Multiple Linear Regression **4**

A.5 L9: Logistic Regression **4**

A.6 L10: Neural Networks **5**

A.7 L11: Language Modeling **6**

A.8 L12: Uncertainty **7**

A.9 Quick Lookup Tables **8**

B.10 Feature Insights at a Glance **10**

B.11 Training Fundamentals **10**

B.12 Training Curve Patterns **12**

B.13 Linear Regression Training **13**

B.14 Binary Classification Training **13**

B.15 Multi-Class Classification **14**

B.16 MLP Architecture Design **14**

B.17 Regularization Techniques **15**

B.18 Learning Rate Analysis **15**

B.19 Validation & Data Splitting **16**

B.20 Epochs & Early Stopping **16**

B.21 Feature Preprocessing **16**

B.22 Loss Functions **16**

B.23 Activation Functions **17**

B.24 Model Comparison **17**

B.25 Evaluation Metrics **17**

B.26 Uncertainty & Ensembles **17**

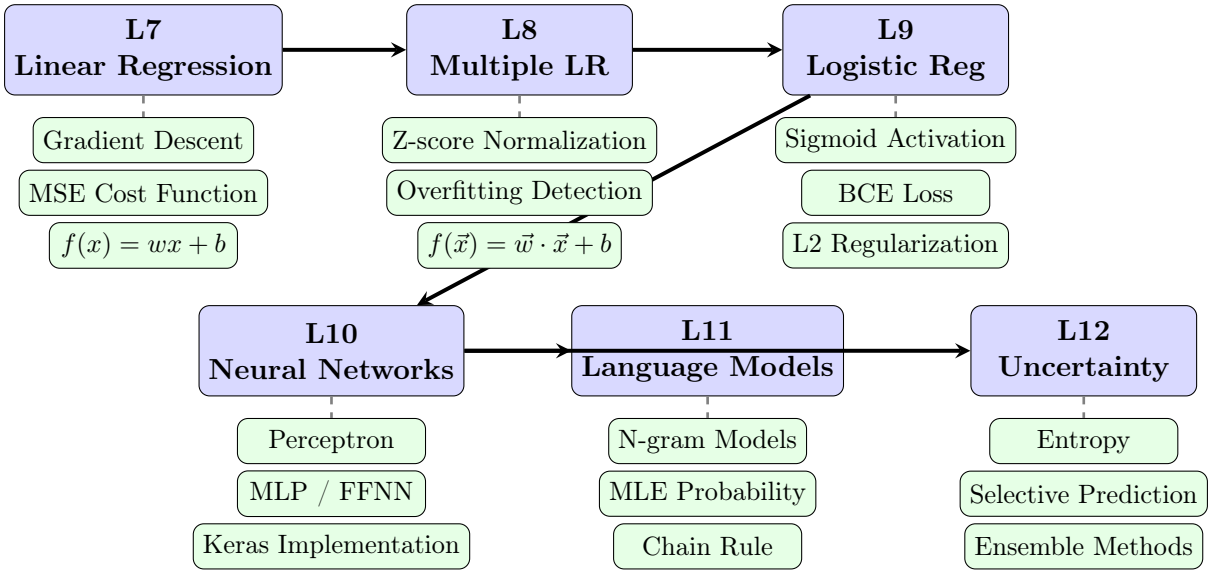
B.27 Language Modeling **18**

B.28 Essential Formulas **18**

B.29 Training Checklist **18**

PART A: Lecture Overview & Key Definitions

A.1 Course Structure: How Lectures Connect



The Red Thread

Regression (L7-L8): Predict continuous values. Learn gradient descent, cost functions, normalization.
Classification (L9): Predict categories. Sigmoid activation, BCE loss, regularization.
Neural Networks (L10): Combine multiple units. Perceptron → MLP with hidden layers.
Applications (L11-L12): Language models use probability. Uncertainty measures confidence.

A.2 Key Terms Dictionary

Fundamental Concepts

Epoch: One complete pass through ALL training examples. After 1 epoch, every sample has been seen once.
Batch: A subset of training data processed together. Batch size = number of samples per gradient update.
Iteration: One gradient update. If 1000 samples with batch size 100, then 10 iterations = 1 epoch.
Learning Rate (α): Step size for weight updates. Controls how much weights change per iteration.
Gradient: Direction of steepest increase in loss. We move opposite to gradient to minimize loss.
Gradient Descent: Algorithm that repeatedly updates weights: $w := w - \alpha \cdot \nabla J(w)$

Model Types

Linear Regression: $f(x) = wx + b$. Predicts continuous output. Uses MSE loss.
Logistic Regression: $f(x) = \sigma(wx + b)$. Binary classification. Uses BCE loss.
Perceptron: Single neuron. Binary output via threshold. Update: if wrong, adjust weights.
MLP (Multi-Layer Perceptron): Neural network with hidden layers. Can learn non-linear boundaries.
FFNN (Feed-Forward Neural Network): Same as MLP. Information flows forward only (no loops).
CNN (Convolutional Neural Network): Specialized for images. Uses filters to detect patterns.

Training Concepts

Training Set: Data used to learn weights (typically 70-80%).
Validation Set: Data used to tune hyperparameters and monitor overfitting (10-20%).
Test Set: Data used ONCE for final evaluation. Never tune based on test results.
Overfitting: Model memorizes training data, fails on new data. Train acc high, val acc low.
Underfitting: Model too simple to capture patterns. Both train and val accuracy low.
Regularization: Technique to prevent overfitting. L2 adds penalty $\frac{\lambda}{2m}\sum w_j^2$ to cost.
Oscillation: When the loss goes up and down instead of steadily decreasing during training. Usually caused by a learning rate that is too high. Solution: reduce the learning rate.

Activation Functions

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$. Output in $(0, 1)$. Used for binary output.
ReLU: $\max(0, z)$. Output in $[0, \infty)$. Default for hidden layers.
Softmax: $\frac{e^{z_i}}{\sum_j e^{z_j}}$. Outputs sum to 1. Used for multi-class output.

Loss Functions

MSE (Mean Squared Error): $\frac{1}{2m}\sum(f(x) - y)^2$. For regression.
BCE (Binary Cross-Entropy): $-[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$. For binary classification.
CE (Categorical Cross-Entropy): $-\sum_k y_k \log(\hat{y}_k)$. For multi-class classification.

Uncertainty Concepts

Entropy: $H = -\sum p_i \log_2(p_i)$. Measures uncertainty. $H = 0$ means certain, $H = \log_2(K)$ means maximum uncertainty.
Selective Prediction: Only predict when confidence exceeds threshold. Trade accuracy for coverage.
Ensemble: Combine multiple models. Average their predictions to reduce variance.

A.3 L7: Linear Regression

What You Learn

Goal: Predict continuous values (e.g., house price from size).
Model: $f_{w,b}(x) = wx + b$ where w = weight, b = bias.
Cost Function (MSE):

$$J(w, b) = \frac{1}{2m}\sum_{i=1}^{m}(f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Why $\frac{1}{2m}$? The $\frac{1}{2}$ makes gradient cleaner (cancels with derivative of square).

Gradient Descent Algorithm

Repeat until convergence:

$$w := w - \alpha\frac{\partial J}{\partial w} = w - \alpha \cdot \frac{1}{m}\sum_{i=1}^{m}(f(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

$$b := b - \alpha\frac{\partial J}{\partial b} = b - \alpha \cdot \frac{1}{m}\sum_{i=1}^{m}(f(x^{(i)}) - y^{(i)})$$

Key Insight: Gradient points to steeper loss. Move opposite direction.

Key Takeaways L7

- Gradient descent finds optimal w, b by minimizing cost
- Learning rate α controls step size (too big = overshoot, too small = slow)
- Cost should decrease each iteration (if not, check α)
- Single feature: one weight w

A.4 L8: Multiple Linear Regression

What You Learn

Goal: Multiple features (e.g., house price from size, bedrooms, age, floors).
Model: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = w_1x_1 + w_2x_2 + ... + w_nx_n + b$
Notation: $\vec{x}^{(i)}$ = feature vector for example i, $x_j^{(i)}$ = feature j of example i

Z-Score Normalization

Problem: Features on different scales cause slow/unstable convergence.
Solution: Normalize each feature to mean=0, std=1:

$$x_{norm} = \frac{x - \mu}{\sigma}$$

Critical Rules:
1. Compute μ, σ from TRAINING data only
2. Store μ, σ for later use
3. Apply SAME μ, σ to validation and test data

Never compute μ, σ on test data (data leakage!)

Overfitting Detection

Polynomial Features: $x \to [x, x^2, x^3, ...]$ increases model complexity.
Problem: High-degree polynomials fit training data perfectly but fail on new data.
Solution: Monitor validation cost. If train cost keeps decreasing but val cost increases $\to$ overfitting!

Key Takeaways L8

- Multiple features use dot product: $\vec{w} \cdot \vec{x}$
- ALWAYS normalize features before training
- Split data: Train/Val/Test to detect overfitting
- Simple models generalize better (Occam's Razor)

A.5 L9: Logistic Regression

What You Learn

Goal: Binary classification (e.g., spam/not spam, tumor malignant/benign).
Model: $f_{\vec{w},b}(\vec{x}) = \sigma(\vec{w} \cdot \vec{x} + b)$
Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties: Output always in $(0, 1)$. Interpret as probability $P(y = 1|x)$.

Binary Cross-Entropy Loss

Why not MSE? For classification, MSE creates non-convex cost with many local minima.
BCE Loss:

$$J = -\frac{1}{m} \sum_{i=1}^{m} [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Intuition:

- If $y = 1$: cost $= -\log(\hat{y})$ (penalize low predictions)
- If $y = 0$: cost $= -\log(1 - \hat{y})$ (penalize high predictions)

L2 Regularization

Purpose: Prevent overfitting by penalizing large weights.

Regularized Cost:

$$J_{reg} = J_{original} + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

Regularized Gradient:

$$\frac{\partial J}{\partial w_j} = \frac{\partial J_{orig}}{\partial w_j} + \frac{\lambda}{m} w_j$$

Note: Bias b is NOT regularized (only weights w_j).

λ **effect**: Larger λ = smaller weights = simpler model = less overfitting (but risk underfitting).

Typical λ Values and When to Use Them

λ Value	When to Use	Effect on Model
0	No regularization needed	Weights can grow large
0.0001	Very mild regularization	Almost no constraint
0.001	Light regularization	Slight weight shrinkage
0.01	**Good starting point**	**Balanced constraint**
0.1	Strong regularization	Significant weight shrinkage
1.0	Very strong	Weights forced very small
10.0	Extreme (rarely used)	Model becomes too simple

Keras syntax: `kernel_regularizer=regularizers.L2(0.01)`

How to choose λ:

1. Start with $\lambda = 0.01$
2. If still overfitting (train acc >> val acc): try $\lambda = 0.1$
3. If underfitting (both acc low): try $\lambda = 0.001$ or $\lambda = 0$
4. Common search: $[0.0001, 0.001, 0.01, 0.1, 1.0]$

Key Takeaways L9

- Sigmoid squashes output to probability
- Decision boundary: predict 1 if $f(x) \geq 0.5$
- BCE loss is convex (guaranteed to find global minimum)
- L2 regularization: use when overfitting (high train acc, low val acc)

A.6 L10: Neural Networks

Perceptron

What: Single artificial neuron. Simplest neural unit.

Model: $f(\vec{x}) = \text{step}(\vec{w} \cdot \vec{x} + b)$

Decision: Output 1 if $\vec{w} \cdot \vec{x} + b > 0$, else 0.

Learning Rule: If prediction wrong:

- If should be 1 but predicted 0: $w := w + \alpha \cdot x$
- If should be 0 but predicted 1: $w := w - \alpha \cdot x$

Limitation: Can only learn linearly separable patterns (fails on XOR).

MLP / FFNN

Solution to Perceptron Limitation: Stack multiple layers.

Architecture:

- **Input layer**: Receives features (not counted as layer)
- **Hidden layers**: Learn intermediate representations. Use ReLU activation.
- **Output layer**: Produces final prediction. Sigmoid (binary) or Softmax (multi-class).

Why "Hidden"? We don't directly observe their outputs during training.

Universal Approximator: MLP with 1+ hidden layer can approximate any continuous function.

Keras Pattern

```
model = keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(64, activation="relu"),
    layers.Dense(32, activation="relu"),
    layers.Dense(1, activation="sigmoid")  # Binary
])
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(X_train, y_train,
    validation_split=0.2,
    epochs=100, batch_size=32)
```

Key Takeaways L10

- Perceptron: single neuron, linear boundary only
- MLP: multiple layers enable non-linear boundaries
- Hidden layers: ReLU activation (avoids vanishing gradient)
- Output layer: Sigmoid (binary), Softmax (multi-class)
- More layers/neurons = more capacity = more overfitting risk

A.7 L11: Language Modeling

What You Learn

Goal: Predict probability of word sequences.
Application: Auto-complete, spell-check, machine translation.
Key Question: Given context words, what's the probability of next word?

N-gram Models

Idea: Approximate probability using previous $n-1$ words.
Bigram ($n = 2$): Condition on previous 1 word.

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Trigram ($n = 3$): Condition on previous 2 words.

$$P(w_i|w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

MLE (Maximum Likelihood Estimation): Count occurrences, divide by context count.

Sentence Probability

Chain Rule: $P(w_1, w_2, ..., w_n) = \prod_i P(w_i|w_1, ..., w_{i-1})$
Bigram Approximation:

$$P(\text{“the cat sat”}) = P(\text{the}) \cdot P(\text{cat}|\text{the}) \cdot P(\text{sat}|\text{cat})$$

Example: If $C(\text{the cat}) = 50$ and $C(\text{the}) = 1000$:

$$P(\text{cat}|\text{the}) = \frac{50}{1000} = 0.05$$

Key Takeaways L11

- N-gram: use last $n-1$ words to predict next
- Bigger n = more context = better but sparse counts
- MLE = count-based probability estimation
- Smoothing needed for unseen word combinations

A.8 L12: Uncertainty

What You Learn

Goal: Know when your model is uncertain about its predictions.

Why Important? High confidence wrong predictions are dangerous. Better to say "I don't know."

Entropy as Uncertainty Measure

Formula:

$$H = -\sum_{i=1}^{K} p_i \log_2(p_i)$$

What are these p_i values? They are the softmax output probabilities. For a 2-class problem, the model outputs something like $[p_{\text{cat}}, p_{\text{dog}}]$ where both sum to 1.

Interpretation:

- $H = 0$: Completely certain (one class has $p = 1$, others have $p = 0$)
- $H = \log_2(K)$: Maximum uncertainty (all classes equally likely)
- 2^H: Effective number of classes the model is "considering"

Understanding Softmax Output Probabilities

What does $[0.9, 0.1]$ mean?

For a cat vs dog classifier:

- $[0.9, 0.1]$ means: 90% confident it's a cat, 10% it's a dog
- $[0.5, 0.5]$ means: 50% cat, 50% dog (model is guessing!)
- $[0.99, 0.01]$ means: 99% confident it's a cat (very sure)

Entropy examples (2 classes):

Output Probabilities	**Entropy H**	**Model is...**
$[1.0, 0.0]$ (100% one class)	$H = 0$	Completely certain
$[0.9, 0.1]$ (90% vs 10%)	$H \approx 0.47$	Quite confident
$[0.7, 0.3]$ (70% vs 30%)	$H \approx 0.88$	Somewhat uncertain
$[0.5, 0.5]$ (50% vs 50%)	$H = 1.0$	Maximum uncertainty

Rule: Lower entropy = more confident. Higher entropy = more uncertain.

Selective Prediction with Confidence Threshold

Idea: Only predict when the model's highest probability exceeds a threshold.

Confidence = the maximum probability in the output = $\max(p_1, p_2, ..., p_K)$

Example: If output is $[0.7, 0.2, 0.1]$ for 3 classes, confidence = 0.7 (70%)

Threshold	**Meaning**	**Result**	**Trade-off**
0.5 (50%)	Predict if >50% sure	More predictions	Lower accuracy
0.7 (70%)	Predict if >70% sure	Medium	Balanced
0.9 (90%)	Predict if >90% sure	Fewer predictions	Higher accuracy

With threshold = 0.8:

- Output $[0.85, 0.15]$: Confidence = $0.85 \geq 0.8$, so PREDICT class 0
- Output $[0.6, 0.4]$: Confidence = $0.6 < 0.8$, so ABSTAIN (don't predict)

Ensemble Methods

Idea: Train multiple models, average their predictions.

Formula:

$$P_{ensemble}(c) = \frac{1}{N}\sum_{i=1}^{N} P_{model_i}(c)$$

Benefits:

- Reduces variance (less overfitting)
- Better uncertainty estimates
- Models disagree on hard/ambiguous examples

Disagreement = Uncertainty: If models give different predictions, the example is likely hard.

Key Takeaways L12

- Entropy quantifies prediction uncertainty
- Low entropy = confident, high entropy = uncertain
- Selective prediction: trade coverage for accuracy
- Ensembles: average models for better predictions and uncertainty

A.9 Quick Lookup Tables

Task to Model Mapping

Task	Model	Output	Loss
Predict price	Linear Reg	Linear	MSE
Yes/No	Logistic Reg	Sigmoid(1)	BCE
Multi-class	MLP	Softmax(K)	CE
Next word	N-gram	Probability	Log-loss

Problem to Solution Mapping

Problem	Solution
Features on different scales	Z-score normalization
Model overfitting	L2 regularization, more data, simpler model
Model underfitting	More neurons/layers, less regularization
Slow convergence	Check learning rate, normalize features
Loss oscillating	Decrease learning rate
Loss stuck high	Increase learning rate or model capacity
Want confidence info	Use entropy, selective prediction
Want better accuracy	Use ensemble of models

Formula Quick Reference

Gradient Descent $w := w - \alpha \frac{\partial J}{\partial w}$

Update weight by stepping opposite to gradient, scaled by learning rate.

Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$

Squashes any input to a value between 0 and 1 (probability).

Softmax $\frac{e^{z_i}}{\sum_j e^{z_j}}$

Converts scores to probabilities that sum to 1 across all classes.

MSE $\frac{1}{2m} \sum (f(x) - y)^2$

Average squared difference between prediction and actual value.

BCE $-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$

Penalizes confident wrong predictions heavily (binary classification).

L2 Cost $J + \frac{\lambda}{2m} \sum w_j^2$

Original cost plus penalty for large weights (prevents overfitting).

Z-score $\frac{x-\mu}{\sigma}$

Center data at 0, scale to unit variance (normalize features).

Entropy $-\sum p_i \log_2(p_i)$

Measures uncertainty: 0 = certain, higher = more uncertain.

Bigram $\frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$

Probability of word given previous word (count-based).

Lecture to Keras Component

Concept	Keras Code
Hidden layer	`Dense(64, activation='relu')`
Binary output	`Dense(1, activation='sigmoid')`
Multi-class output	`Dense(K, activation='softmax')`
L2 regularization	`kernel_regularizer=regularizers.L2(0.01)`
Binary loss	`loss='binary_crossentropy'`
Multi-class loss	`loss='sparse_categorical_crossentropy'`
Optimizer	`optimizer=Adam(learning_rate=0.001)`
Validation	`validation_split=0.2`

PART B: Training Guide & Reference Tables

B.10 Feature Insights at a Glance

Reading Bar Charts for Feature Importance

Goal: Quickly identify which features help predict the target.
Method: Plot feature vs target mean. Look at bar heights.
Informative Feature: Bars have DIFFERENT heights across categories.

- Example: "education" shows high school avg=40K, bachelor's avg=65K, master's avg=85K
- Clear difference = feature helps distinguish outcomes

Non-Informative Feature: Bars have SIMILAR heights across categories.

- Example: "eye color" shows blue avg=52K, brown avg=51K, green avg=53K
- Almost same = feature doesn't help much

Visual Pattern Recognition

What You See	Meaning	Action
Big height differences	Strong predictor	Keep feature
Small height differences	Weak predictor	Consider dropping
Clear trend (up/down)	Linear relationship	Good for model
Random/no pattern	No relationship	May add noise
One bar much higher	Category matters	Keep feature
All bars same height	No discrimination	Less useful

Quick Feature Selection Rules

MOST Informative (keep these):

- Features where target mean varies a lot between categories
- Features showing clear trends (monotonic increase/decrease)
- Features with distinct separation between groups

LEAST Informative (consider removing):

- Features where all categories have similar target mean
- Features with high variance but no clear pattern
- Features with too many missing values

Numeric vs Categorical Features

Categorical (discrete values): Use bar charts, count plots.
Numeric (continuous values): Use scatter plots, correlation.
Tip: Bar charts make it easy to compare category means at a glance.

B.11 Training Fundamentals

Core Terminology

Epoch: One complete pass through all training data.
Batch Size: Number of samples per gradient update.
Learning Rate (α): Step size for weight updates.
Validation Split: Fraction held out for monitoring.
Gradient: Direction of steepest loss increase.
Loss/Cost (J): Measures prediction error.

When You INCREASE Each Parameter

Increase...	What Happens	Risk
Learning Rate	Faster training	Divergence
Batch Size	Faster per epoch	Worse generalization
Epochs	More learning	Overfitting
Regularization (λ)	Smaller weights	Underfitting
Hidden Neurons	Higher capacity	Overfitting
Hidden Layers	More abstraction	Harder to train

Hyperparameter Starting Values

Parameter	Start	Range	Adjust If...
Learning Rate	0.001	0.0001–0.1	Oscillating/slow
Batch Size	32	16–256	Memory/speed
Epochs	100	50–500	Under/overfit
Val Split	0.2	0.1–0.3	Dataset size
L2 λ	0.01	0.001–0.1	Overfitting
Hidden Neurons	64	32–256	Capacity
Hidden Layers	1	1–3	Complexity

Dataset Size → Hyperparameter Configuration

Binary Classification by Dataset Size

Parameter	Small (<1K)	Med (1K-10K)	Large (10K-100K)	V.Large (>100K)
Learning Rate	0.001	0.001	0.001–0.01	0.01
Batch Size	16	32	64–128	128–256
Epochs	50–100	100–200	50–100	20–50
Val Split	0.3	0.2	0.15	0.1
L2 λ	0.1	0.01	0.001–0.01	0.001
Hidden Neurons	16–32	32–64	64–128	128–256
Hidden Layers	1	1–2	2	2–3

Key insights:
- Small data → high λ (0.1), few neurons, large val split (30%)
- More data → can reduce regularization, add capacity
- Large batch = faster training but may generalize worse
- Fewer epochs needed when dataset is large (sees more per epoch)
- Always start simple, add complexity only if underfitting

MLP (Multi-Class) by Dataset Size

Parameter	Small (<1K)	Med (1K-10K)	Large (10K-100K)	V.Large (>100K)
Learning Rate	0.0001–0.001	0.001	0.001	0.001–0.01
Batch Size	16	32	64–128	256
Epochs	100–200	100–150	50–100	10–30
Val Split	0.3	0.2	0.15	0.05–0.1
L2 λ	0.1–0.5	0.01–0.1	0.001–0.01	0.0001–0.001
Hidden Neurons	32	64–128	128–256	256–512
Hidden Layers	1	1–2	2–3	3+
Output Activ.	softmax	softmax	softmax	softmax

Key insights:
- Multi-class needs lower LR than binary (more complex optimization)
- Small data + many classes = very high overfit risk → strong λ
- Large data allows 3+ layers and 256+ neurons per layer
- Softmax output always for multi-class (probabilities sum to 1)
- With >100K samples, can use aggressive batch sizes (256) for speed

Linear Regression by Dataset Size

Parameter	Small (<1K)	Med (1K-10K)	Large (10K-100K)	V.Large (>100K)
Learning Rate	0.001	0.001–0.01	0.01	0.01–0.1
Batch Size	16	32	64–128	128–256
Epochs	100–200	100–150	50–100	20–50
Val Split	0.3	0.2	0.15	0.1
L2 λ	0.01–0.1	0.001–0.01	0.001	0.0001–0.001
Hidden Neurons	16–32	32–64	64–128	128–256
Hidden Layers	1	1–2	2	2–3
Output Activ.	linear	linear	linear	linear
Loss Function	mse	mse	mse	mse

Key insights:
- Regression tolerates higher LR than classification (simpler loss surface)
- Output is always `linear` (no activation) for unbounded predictions
- MSE loss standard; use MAE if many outliers
- Small data → regularization prevents overfitting to noise
- Larger batch sizes OK for regression (less noisy gradients needed)

Dataset Size Rules of Thumb

Small data (<1K samples):
- High overfitting risk → strong regularization ($\lambda = 0.1$)
- Keep model simple (1 hidden layer, few neurons)
- Use larger validation split (30%) for reliable estimates
- Consider simpler models (Logistic Regression) first

Medium data (1K–10K samples):
- Balanced approach: moderate regularization ($\lambda = 0.01$)
- Can use 1–2 hidden layers with 64 neurons
- Standard validation split (20%)

Large data (>10K samples):
- Lower overfitting risk → less regularization needed
- Can use deeper networks (2–3 layers, 128+ neurons)
- Smaller validation split OK (10–15%)
- Larger batch sizes for faster training

B.12 Training Curve Patterns

Loss Curve Diagnosis

You See...	Problem	Fix
Both losses decrease	Good!	Keep going
Train decreases, Val increases	Overfitting	Add regularization
Both stuck high	Underfitting	Increase model size
Wild oscillation	LR too high	Divide LR by 10
Very slow decrease	LR too low	Multiply LR by 10
Loss becomes NaN	Exploding gradient	Lower LR, check data

Accuracy-Based Diagnosis

Train	Val	Status	Fix
Low	Low	Underfitting	Bigger model, less λ, more epochs
High	Low	Overfitting	More λ, smaller model, more data
High	High	Good fit	Ready for test set
Low	High	Bug	Check your code/data

Convergence Indicators

Converged: Loss stops decreasing, train and val accuracy are similar.
Not converged: Loss is still going down noticeably.
Early stopping: Stop when validation loss stops improving.
Patience: Number of epochs to wait before stopping (typically 5-20).

B.13 Linear Regression Training

Model Setup

Model: $f(\vec{x}) = \vec{w} \cdot \vec{x} + b$
Loss: Mean Squared Error (MSE)

$$J = \frac{1}{2m} \sum_{i=1}^{m} (f(\vec{x}^{(i)}) - y^{(i)})^2$$

Output: 1 neuron + linear activation (no activation)
Prediction: Direct output value (no threshold needed)

Linear Regression Architecture

`Input(n)` → `Dense(64, relu)` → `Dense(32, relu)` → `Dense(1, linear)`

Compile: `loss='mse', optimizer=Adam(lr=0.001), metrics=['mae']`

Linear Regression Parameters

Component	Value	Reason
Output neurons	1	Single continuous value
Output activation	linear	Unbounded output range
Loss function	MSE	Penalizes squared error
Alternative loss	MAE	Robust to outliers
Metrics	mae	Interpretable error unit

When to Use MAE vs MSE

MSE (Mean Squared Error): Default choice. Penalizes large errors more heavily.
- Use when: Large errors are especially bad
- Sensitive to outliers (squares blow up)

MAE (Mean Absolute Error): More robust alternative.
- Use when: Data has outliers you can't remove
- Treats all errors equally (no squaring)

Rule: Start with MSE. If results are bad and you have outliers, try MAE.

B.14 Binary Classification Training

Model Setup

Model: $f(\vec{x}) = \sigma(\vec{w} \cdot \vec{x} + b)$
Loss: Binary Cross-Entropy (BCE)

$$L = -\frac{1}{m} \sum_{i=1}^{m} [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Output: 1 neuron + sigmoid activation
Decision: Predict 1 if $f(\vec{x}) \geq 0.5$, else 0

Binary Classification Architecture

`Input(n)` → `Dense(64, relu)` → `Dense(32, relu)` → `Dense(1, sigmoid)`

Compile: `loss='binary_crossentropy', optimizer=Adam(lr=0.001)`

Binary Classification Parameters

Component	Value	Reason
Output neurons	1	Binary output
Output activation	sigmoid	Maps to $[0, 1]$
Loss function	BCE	Penalizes wrong conf.
Threshold	0.5	Decision boundary
Metrics	accuracy	Primary metric

B.15 Multi-Class Classification

Model Setup

Output: K neurons + softmax activation
Softmax: $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
Loss: Categorical Cross-Entropy

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

Sparse vs Dense Labels:
- Integer labels (0,1,2,...): `sparse_categorical_crossentropy`
- One-hot labels: `categorical_crossentropy`

Multi-Class Architecture

Compile: `loss='sparse_categorical_crossentropy'`

Multi-Class Parameters

Component	Value	Reason
Output neurons	K (classes)	One per class
Output activation	softmax	Prob. distribution
Loss function	CE	Multi-class
Prediction	argmax	Highest prob. class

B.16 MLP Architecture Design

Architecture Decision Matrix

Decision	Options	Trade-off
Hidden layers	1–3	Abstraction vs training
Neurons/layer	32–256	Capacity vs overfit
Hidden activation	ReLU	Default choice
Output activation	sigmoid/softmax	Task-dependent
Regularization	L2, Dropout	Overfit prevention

Layer Design Guidelines

Width (neurons per layer): More neurons = higher capacity but more overfitting risk.
Depth (number of layers): More layers = more abstract features but harder to train.
Starting point: 1-2 hidden layers, 64-128 neurons each.
Common Patterns:
- Funnel: 128, then 64, then 32, then output
- Constant: 64, then 64, then output
- Pyramid (rare): 32, then 64, then output

Model Size vs Performance

Architecture	Params	Train Acc	Val Acc
Dense(16)	160	Medium	Medium
Dense(64)	650	High	High (good)
Dense(256)	2500	Very High	Lower (overfit)
Dense(64,64)	4500	Very High	Low (overfit)

Rule: More parameters needs more data OR more regularization.

B.17 Regularization Techniques

L2 Regularization (Weight Decay)

Modified Cost:

$$J_{reg} = J_{original} + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

Modified Gradient:

$$\frac{\partial J}{\partial w_j} = \frac{\partial J_{orig}}{\partial w_j} + \frac{\lambda}{m} w_j$$

Effect: Shrinks weights toward zero.
Note: Bias b is NOT regularized.

λ Effect on Training

λ Value	Weights	Train Acc	Val Acc
0 (none)	Large	High	Low (overfit)
0.001 (small)	Medium	High	Medium
0.01 (good start)	Moderate	Medium-High	High
0.1 (too high)	Very small	Low	Low (underfit)

Goal: Find λ where Validation Accuracy is highest.

Regularization Diagnosis

λ too low: Train accuracy much better than Val accuracy.
Solution: Increase λ to reduce overfitting.
λ too high: Both accuracies are low.
Solution: Decrease λ to allow more learning.
Just right: Train and Val accuracy are close, both high.

B.18 Learning Rate Analysis

Learning Rate Behavior

LR Value	Loss Curve Looks Like	What To Do
0.1+ (too high)	Oscillates or explodes	Divide by 10
0.01 (high)	Fast but unstable	Monitor closely
0.001 (good)	Smooth decrease	Keep this value
0.0001 (low)	Very slow decrease	Multiply by 10
1e-5 (too low)	Almost flat/no change	Multiply by 100

LR Interactions with Other Parameters

LR + Batch Size: If you double batch size, often need to double LR too.
LR + Epochs: Lower LR needs more epochs to converge. Higher LR needs fewer epochs but is unstable.
LR + Regularization: Strong regularization (λ) slows learning. May need more epochs when λ is high.

B.19 Validation & Data Splitting

Data Split Strategy

Dataset Size	Val Split	Reasoning
Small (<1K)	0.3	Need reliable val estimate
Medium (1K-10K)	0.2	Balance train/val
Large (>10K)	0.1-0.15	More data for training
Very Large (>100K)	0.05-0.1	Enough for both

Three-Way Split

Training: Used for learning weights (60-80%)
Validation: Used for tuning hyperparameters (10-20%)
Test: Used ONCE for final evaluation (10-20%)
Golden Rule: NEVER tune based on test performance!

B.20 Epochs & Early Stopping

Epoch Effects

Epochs	Train Loss	Val Loss	Status
10	High	High	Underfit
50	Medium	Medium	Learning
100	Low	Low	Good fit
200	Very Low	Rising	Overfit!

Early Stopping Protocol

Monitor: Validation loss (not training loss)
Patience: Number of epochs without improvement
Best model: Save weights when val loss is lowest
Typical patience: 5-20 epochs
Stop when: Val loss hasn't improved for `patience` epochs

B.21 Feature Preprocessing

Z-Score Normalization

Formula: $x_{norm} = \frac{x-\mu}{\sigma}$
Result: Mean=0, Std=1
Implementation:
1. Compute μ, σ from **training data only**
2. Store μ, σ for later use
3. Apply to train, val, and test data

Normalization Impact

Setting	Convergence	LR Needed
No normalization	Slow/fails	Very small
With normalization	Fast	Standard (0.001)

Why? Features on different scales cause gradient descent to oscillate.

B.22 Loss Functions

Loss by Task

Task	Loss	Output
Regression	MSE	Linear
Binary Class.	BCE	Sigmoid(1)
Multi-Class	CE	Softmax(K)

Loss Formulas

MSE (Regression): $J = \frac{1}{2m}\sum_{i=1}^{m}(f(x^{(i)}) - y^{(i)})^2$
BCE (Binary): $L = -[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$
CE (Multi-class): $L = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$

B.23 Activation Functions

Activation Comparison

Function	Formula	Range	Use
ReLU	$\max(0, z)$	$[0, \infty)$	Hidden
Sigmoid	$\frac{1}{1+e^{-z}}$	$(0, 1)$	Binary out
Softmax	$\frac{e^{z_i}}{\sum e^{z_j}}$	$(0, 1)$, sum=1	Multi out
Linear	z	$(-\infty, \infty)$	Regression

Activation Selection Rule

Hidden layers: Always use ReLU (fast, no vanishing gradient).

Output layer depends on task:

- Binary classification: use Sigmoid (1 neuron)
- Multi-class classification: use Softmax (K neurons)
- Regression: use Linear/None

B.24 Model Comparison

Logistic Regression vs MLP

Aspect	LogReg	MLP
Decision boundary	Linear only	Any shape
Parameters	$n + 1$	Thousands+
Interpretability	High	Low
Overfitting risk	Low	High
Data needed	Less	More
Training speed	Fast	Slower

Use LogReg as baseline, then try MLP if needed.

B.25 Evaluation Metrics

Classification Metrics

Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$ (bad for imbalanced)

Precision: $\frac{TP}{TP+FP}$ (of predicted +, how many correct?)

Recall: $\frac{TP}{TP+FN}$ (of actual +, how many found?)

F1: $\frac{2 \cdot Prec \cdot Rec}{Prec+Rec}$ (use for imbalanced)

Imbalanced Data

If 99% negative: predicting "always negative" = 99% accuracy but 0% recall!

Solution: Use F1-score or balanced accuracy.

B.26 Uncertainty & Ensembles

Entropy as Uncertainty

Formula: $H = -\sum_{i=1}^{K} p_i \log_2(p_i)$

Interpretation:

$H = 0$	Certain (one class $p = 1$)
$H = \log_2(K)$	Max uncertain (uniform)
2^H	Effective # of classes

Selective Prediction

Idea: Only predict when model is confident enough.

Threshold: Minimum confidence to make prediction.

Coverage: Fraction of data where model makes prediction.

Threshold	Accuracy	Coverage
Low (0.5)	Lower accuracy	More predictions
High (0.9)	Higher accuracy	Fewer predictions

Trade-off: Higher threshold = better accuracy but fewer predictions.

Ensemble Averaging

Formula: $P_{ensemble}(c) = \frac{1}{N}\sum_{i=1}^{N} P_{model_i}(c)$
Benefits: Reduces variance, better uncertainty, models disagree on hard examples.

B.27 Language Modeling

N-gram Models

Bigram MLE: $P(w_i|w_{i-1}) = \frac{C(w_{i-1},w_i)}{C(w_{i-1})}$
Trigram MLE: $P(w_i|w_{i-2}, w_{i-1}) = \frac{C(w_{i-2},w_{i-1},w_i)}{C(w_{i-2},w_{i-1})}$
Sentence Prob: $P(S) = \prod_i P(w_i|context)$

B.28 Essential Formulas

Quick Reference

Gradient Descent: $w := w - \alpha\frac{\partial J}{\partial w}$
Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
Softmax: $\frac{e^{z_i}}{\sum_j e^{z_j}}$
ReLU: $\max(0, z)$
MSE: $\frac{1}{2m}\sum(f(x) - y)^2$
BCE: $-[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$
L2 Reg: $J + \frac{\lambda}{2m}\sum w_j^2$
Z-score: $\frac{x-\mu}{\sigma}$
Entropy: $-\sum p_i \log_2(p_i)$
Bigram: $\frac{C(w_{i-1},w_i)}{C(w_{i-1})}$

B.29 Training Checklist

Step-by-Step Training

1. **Preprocess**: Normalize features (Z-score)
2. **Split**: Train/Val/Test (70/15/15 or 80/10/10)
3. **Start simple**: 1 layer, 64 neurons, ReLU
4. **Set hyperparams**: LR=0.001, batch=32, epochs=100
5. **Train**: Monitor train & val loss curves
6. **Diagnose**: Check for over/underfit
7. **Tune**: Adjust ONE param at a time
8. **Add regularization**: If overfitting (L2 λ=0.01)
9. **Final test**: Evaluate ONCE on test set